CST1500 – Laboratory Worksheet - Week 7

These weekly laboratory worksheets provide information and exercises to complete during class or at your own leisure. The practical exercises will assist you with your coursework as well as provide you with valuable knowledge and skills. Please do your best to keep up with the weekly laboratory worksheets in order to maximise your potential for good marks and satisfactory progress in the module.

Exercises are denoted in red writing. For example:

EX 1: Identification of PC parts (identify as many as you can).

Contents

Lectures videos to watch this week 1

Coursework 2 1

Introduction to Python 1

Lectures videos to watch this week

Lecture 14 Synchronization Tools
Lecture 15 Deadlocks

There will be a FORMATIVE test available next week which will cover lectures 7 -13.

Coursework 2

Details of the Coursework 2 will be presented this week by your lab tutor. The coursework specification can be found on the course page in the Coursework section.

Introduction to Python

Lab objectives

In this lab session, you will be introduced to Python programming language. You will learn the following:

- Python basics
  - How to run a Python program
  - How to write a Python program
- Variables
- Basic data types: number, string, list, and dictionary

• Control flow:
  – Branching structure: if, if-else, if-elif-else
  – Loops: for loop and while loop
• Input/ Output
  – Comments & modules
  – Multithreading

Python – Background

Python was released in 1991 by Guido van Rossum. It is an object-oriented, high-level programming language. Opposingly to programming languages like C or BASIC families it does not need to be compiled prior to running. Python is an interpreted programming language – which means you need a Python interpreter to run the code, instead of precompiling it, it will be interpreted as code runs. It has its advantages and disadvantages. Some of the biggest advantages are mobility – you can write code once and run it on any operating system/device that has a Python interpreter and you don't have to compile code every time you change it. One of the main issues is performance, interpreted programming languages tend to be slower than compiled ones.

Python has a very dev-friendly structure which is slightly different to other programming languages you might know. Usually programming languages like C use parenthesis to indicate range of functions of statements, Python uses indent instead (see Fig. 1 Simple C program and Fig. 2 Simple Python program). Please make sure you are consistent with type of indentation in your program if use tabs for indents make sure you use tabs, if use space for indents make sure you use same amount of spaces throughout the whole program.

```
#include <stdio.h>
int main (){
     int i = 0;
     while(i < 5)
     {
          printf("%d", i*i);
          i++;
     }
     return 0;
}
```

Fig 1: A simple C program.

```
i = 1

while i < 5:
     print(i*i)
     i += 1
```

Fig 2: A simple Python program.

As you can see in the above examples Python code does not have any braces (curly brackets {}) to indicate the range of the while statement – it uses indentation instead. You have to be consistent with indentation depth – use single tabs to indicate range of the first statement and one more for each nested statement.

Python is case sensitive; thus, variableTest won't be equal to VariableTest. Make sure you are consistent with naming and follow good programming practices – start names with lower case letters, use capitals to indicate the first initial of the next word in a variable name e.g. myFirstProgram.

There are two versions of Python – Python 2.x and Python 3.x. The structure of the code is different so make sure you use the correct interpreter for the version of your code – if you write code up to Python 3 standards, use a Python 3 interpreter.
There are two main ways of interacting with Python as shown in the following table:

	Interactive mode	Normal mode
Description	Takes single user inputs, evaluates them, and returns the result to the user (**read–eval–print loop** (**REPL**))	Execute a Python **script** on the Unix command prompt
Benefits	• Use as a **sandbox**: explore new features • Easy to write quick **"throw away" scripts** • Useful for debugging • Use it as a calculator!	• Run long complicated programs • The script contains all of the commands

Usage	`$ python3`
	`Python 3.4.0 (default, Apr 11 2014, 13:05:11) [GCC 4.8.2] on linux2 Type "help", "copyright", "credits" or "license" for more information.`
	`>>>`
	This is Python's command prompt. It means, "I'm ready for a command!" Don't type the ">>>"
		`$ python3 <script.py>`

59

Please note: In order to create your Python script, watch this 6 minute informative video:

https://www.youtube.com/watch?v=iFIPQL4yqs0&ab_channel=RajiRajesh

Comments in #Python

Python offers a single line comment feature, unfortunately it does not support block comments, so if you want to comment out a whole block of code you have to put a # (hashtag symbol) in front of each line you want to comment out. There is a way to write multi-line comments however by using three " at the beginning and end of the multi-line comment. Be sure to type the " in the code and not copy and paste it.

```
# This is a single line comment

# This is also a way
# to write comments

"""
This is
a multi-line
comment
"""
```

EX 1: Understanding the Inputting & Outputting Data in Python

Outputting Data

If you want to display something on screen you can use the print() function. An example of code that prints to screen in IDLE is shown below:

```
print("Hello World")
```

The expected output:

```
>>>
Hello World
>>>
```

Inputting Data

If you want the user to enter data into the program, you can use the input() function.

An example of code that will ask the user to enter their name and display it on screen using print() is shown below:

```
# displays the message on screen and stores the input from the
# user in a variable called name

name = input("What is your name? ")

print("Hi " + name)
```

The expected output of the above script is:

```
>>>
What is your name? Bob
Hi Bob
>>>
```

EX 2: More examples of the different inputs with their expected outputs in Python

Example program 1 - Enter a Word

The code for the program below will allow the user to enter a word and store it in a variable called word. It will then use the print() function to output the word that they entered.

```
word = input("Please enter a word ")
print("You entered the word " + word)
```

The expected output of the above script is:

```
>>>
Please enter a word computing
You entered the word computing
>>>
```

Example program 2 - Address Program

The code for the program below will allow the user to enter various pieces of information and store them in different variables. The print() function is then used to output all of the information.

```
number = input("Enter your house number: ")
street = input("Enter your street name: ")
town = input("Enter your town/city: ")
county = input("Enter your county: ")
postcode = input("Enter your postcode: ")
print("\nAddress Details:\n" + "Street: " + number + " " +
street + "\nTown/City: " + town + "\nCounty: " + county +
"\nPostcode: " + postcode)
```

The expected output of the above script is:

```
>>>
Enter your house number: 55
Enter your street name: Frankland Close
Enter your town/city: Wakefield
Enter your county: West Yorkshire
Enter your postcode: WF1 2PF

Address Details:
Street: 55 Frankland Close
Town/City: Wakefield
County: West Yorkshire
Postcode: WF1 2PF
>>>
```

You can concatenate (join together) variables with strings in a print() function. In the address example `print("Street: " + number + " " + street + "\nTown/City: " + town)` will combine the strings “Street” and “Town/City” with the variables number, street and town. `\n` is used to start a new line when it is displayed on screen.

Variables

The most basic component of any programming language are "things," also called variables. A variable has a name and an associated value. Variables in Python don’t need to be declared prior to its initiation, they are not bond to data types; thus, variable

that initially stores an integer can store a String later on. Variables are defined and initialised when you assign a value to it (see below).

```
x = 1
x = 1.0
x = "Hey I am a String"
x = ["String1", "String2", "String3"]
```

As you can see in the example above, the variable was initially declared as an integer, then it became a floating point number, then a String, and finally x became a List.

Because you don't have to change the type of a variable prior to its redeclaration, it is much simpler to write code.

To assign a value to a variable (save a variable) use =

>>> x = 2

To determine what type of variable, use the type function

>>> type(x)
IMPORTANT: the variable name must be on the left-hand side of the =

>>> x = 2 Correct

>>> 2 = x Incorrect

Variable naming best practices (conventions)

• Must start with a letter
• Can contain letters, numbers, and underscores - no spaces!
• Python is case-sensitive: x ≠ X
• Variable names should be descriptive and have a reasonable length
• Use ALL CAPS for constants, e.g., PI
• Do not use names already reserved for other purposes (min, max, int)

EX 3: Understanding the concept of naming variables and their types in Python

For example, all of the following are valid variable names:

>>> name = "Bob"
>>> Age = 54
>>> has_W2 = True
>>> print(name, Age, has_W2)

Expected output

```
Bob 54 True
```

But this one is not, because a variable name can't begin with a digit:

>>> 1099_filed = False

Expected output

```
SyntaxError: invalid token
```

Create some variables of different types and provide some suitable and unsuitable names to test which are acceptable, and which cause errors.

Casting Variables

Python will automatically decide what type of data a variable should be, sometimes this isn't right, therefore you will have to convert/cast variables to a different data type.

EX 4: Examples of casting variables to a different data type in Python

Integer

The program below will multiply a number by 5. When data is input from the user, it will store it as a string. You will need to cast the variable number to an integer before performing a calculation. An example of how you do this is shown below:

```
number = input("Enter a whole number ")
#converts the variable number to an integer and multiplies it
by 5.
answer = int(number) * 5
print(answer)
```

What is the expected output?

Real / Float

The program below will ask the user to enter their weight in kilograms (this could be a decimal) and convert it to pounds. You will need to cast the variable kg to a float before converting it. An example of how you do this is shown below:

```
kg = input("Enter the weight in KG ")
81
pounds = float(kg) * 2.2
print(pounds)
```

What is the expected output?

String

Using the program above, if you wanted to improve the print message so that it said *'Your weight in pounds is X'* you would need to cast the variable pounds to a string as it is currently a float.

```
print("Your weight in pounds is " + pounds)
```

This is what would happen if you ran the code currently:

```
>>>
Enter the weight in KG 43.4
Traceback (most recent call last):
  File "H:/Sample/kg.py", line 3, in <module>
    print("Your weight in pounds is " + pounds)
TypeError: Can't convert 'float' object to str implicitly
>>>
```

This is because the variable pounds is a float and to be joined with a string in the print message it needs to be converted to a string, the code is below:

```
print("Your weight in pounds is " + str(pounds))
```

Now when run the code it will work correctly:

```
>>>
Enter the weight in KG 43.4
Your weight in pounds is 95.48
>>>
```

EX 5: Performing advanced program in Python for capacity calculation

Write a Python script/code that will allow the user to enter the height, width and depth of a water tank, then calculate and output the capacity. Please note: To calculate the capacity; the following formula needs to be considered:

capacity = (height * width * depth) / 1000

The expected output will look like the following:

```
>>>
Enter the tank height (cm): 45
Enter the tank width (cm): 23
Enter the tank depth (cm): 56
The tank holds 57.96 litres of water
>>>
```

To round a variable you may use the round() function. You write the name of the variable followed by the number of decimal places e.g. round(capacity,2). The code is written as follows:

```
#outputs the capacity of the water tank
print("The tank holds" + round(capacity,2) + "litres of water")
```

EX 6: Creating a complex program in Python to calculate volume

Write a Python script/code that will allow the user to enter the radius and height of a circle, then calculate and output the cylinder volume and surface area. Please note: To calculate the volume and surface area; the following formulas need to be considered:

volume = pie * (radius*radius) * height
surface = (2*(pie*(radius*radius))) + (2*(pie*radius*height))
The expected output will look like this:

```
>>>
Enter the cylinder radius: 2.3
Enter the cylinder height: 4.6
The volume of your cylinder is 76.45 to 2 decimal places
The surface area of your cylinder is 99.71 to 2 decimal places
>>>
```

To round a variable you may use the round() function as stated earlier. You write the name of the variable followed by the number of decimal places e.g. round(volume,2).

The code is as follows:

```
#outputs the volume and surface area of the cylinder
print("The volume of your cylinder is", round(volume,2), "to 2 decimal places")

print("The surface area of your cylinder is", round(surface,2),"to 2 decimal places")
```